Aufbau

1. Motivation
2. Formalisierung erarbeiten
3. Soundness & Completeness Proofs (Sketch?)
4. Deadlock Algorithm

Keywords

- **Message Passing**: Query, Response, Cast
- **Remote Procedure Calls**
- **Deadlock**: Cyclic wait
- **Monitors**: Analyse traffic
  - Probes: Communication with other monitors
- **Black-Box**: No introspection

Quirks

- The monitored path might be different!

Proofs and Definitions

- Criterion 2.1 (Monitor Instrumentation Transparency)
  - Operational Correspondance vs. Simulation Relation
  - soundess allows some extra steps
- Criterion 2.2 (Preciseness of Deadlock Detection)
  - completeness allows some extra steps
- Lock
- Deadlock
- Deadlockset
- Lemma 3.8 (Persistence of deadlocks).

Todo

- Probing
- Process Calculi?
- SRPC relation to RPC?

Correct Black-Box Monitors for Distributed Deadlock Detection: Formalisation and Implementation

Disecting the Title

- We are given a network setting with communicating RPC Services
- **RPC**: Remote Procedure Call (Query, Response, Cast)
- **Deadlock**: Cyclic waiting for resources in such a network
- **Distributed**: No central authority (due to performance)
- **Blackbox**: No introspection of services, only communication behaviour
- **Monitors**: Wrappers around service that observe communication

Contributions

1. A formal model for RPC services and communication
2. A formal method to install distributed, black-box, outline monitors
3. A Monitoring algorithm that detects deadlock in a sound and complete way and proof its soundness
4. Implementation of DDMon (for Erlang/OTP)

Agenda

1. Formalising Processes & Services
2. Formalising Networks
3. Installing Monitors
4. Proof for Deadlock-detection
5. Deadlock Detection Algorithm
6. Proof of Soundness & Completeness

Wanted Properties

1. **Distributed**: No centralized component
2. **Blackbox and Outline**: Detect deadlocks by just observing the incoming and outgoing messages of the service they monitor
3. **Transparent**: Don’t alter the execution
4. **Precise**: Detect Deadlock iff it existst

Nodes

Formalism: Services

- We define *Single-threaded Remote Procedure Calls* (SRPC)

$$\begin{array}{rlrl} \text{Communication tag} & t := Q \mid R \mid C & \text{Name} & n := n_0 \mid n_1 \mid \ldots \\ \text{Client} & n^{\perp} := n \mid \perp & \text{Process} & P := \ldots \\ \text{Queue} & q := \varepsilon \mid n(t) \mid q + q & \text{Service} & S := \langle q^i \mid P \mid q^o \rangle \end{array}$$

- The Behaviour of P must be compatible with an **abstract SRPC Process G**

Formalism: LTS Semantic of abstract SRPC Process G

- We define the possible states of an abstract Process G. The LTS Semantics are defined in Figure 1 using the transitions labels γ.

$$G := \text{Ready} \mid \text{Working}(n_c^{\perp}) \mid \text{Locked}(n_c^{\perp}, n_s)$$

$$\gamma :=? n(t) \mid !n(t) \mid \tau$$

$$\frac{}{\text{Ready} \xrightarrow{?n_c^{\perp}(\text{Q})} \text{Working}(n_c^{\perp})}\ \text{SRPC-RQ} \qquad \frac{}{\text{Working}(n_c^{\perp}) \xrightarrow{\tau} \text{Working}(n_c^{\perp})}\ \text{SRPC-WT}$$

$$\frac{n_c \neq \perp}{\text{Working}(n_c) \xrightarrow{!n_c(\text{R})} \text{Ready}}\ \text{SRPC-WR} \qquad \frac{}{\text{Working}(\perp) \xrightarrow{\tau} \text{Ready}}\ \text{SRPC-WX}$$

$$\frac{}{\text{Working}(n_c^{\perp}) \xrightarrow{!n(\text{C})} \text{Working}(n_c^{\perp})}\ \text{SRPC-WC} \qquad \frac{}{\text{Working}(n_c^{\perp}) \xrightarrow{!n_s(\text{Q})} \text{Locked}(n_c^{\perp}, n_s)}\ \text{SRPC-WQ}$$

$$\frac{}{\text{Locked}(n_c^{\perp}, n_s) \xrightarrow{?n_s(\text{R})} \text{Working}(n_c^{\perp})}\ \text{SRPC-L}$$

Figure 1: LTS semantics of the abstract SRPC process G

Formalism: LTS Semantic of Services

- We extend a process P with queues and search operations to form a service S. The lts-semantics in Figure 2 are using the transition labels in β. δ is a transition label for queues.

$$\delta := ![n(t)]$$

$$\beta :=? n(t) \mid !n(t) \mid \tau(? n(t)) \mid \tau(!n(t)) \mid \tau(\tau)$$

$$\frac{}{n(t) \mathbin{+\!\!+} q \xrightarrow{![n(t)]} q}\ \textsc{QUE-FIND} \qquad \frac{q_1^i = n(t) \mathbin{+\!\!+} q_0^i}{\langle q_0^i | P | q^o \rangle \xrightarrow{?n(t)} \langle q_1^i | P | q^o \rangle}\ \textsc{SER-I} \qquad \frac{q_0^o = n(t) \mathbin{+\!\!+} q_1^o}{\langle q^i | P | q_0^o \rangle \xrightarrow{!n(t)} \langle q^i | P | q_1^o \rangle}\ \textsc{SER-O} \qquad \frac{P_0 \xrightarrow{\tau} P_1}{\langle q^i | P_0 | q^o \rangle \xrightarrow{\tau(\tau)} \langle q^i | P_1 | q^o \rangle}\ \textsc{SER-TT}$$

$$\frac{n(t) \neq n'(t') \quad q \xrightarrow{![n'(t')]} q'}{n(t) \mathbin{+\!\!+} q \xrightarrow{![n'(t')]} n(t) \mathbin{+\!\!+} q'}\ \textsc{QUE-SEEK} \qquad \frac{q_0^i \xrightarrow{![n(t)]} q_1^i \quad P_0 \xrightarrow{?n(t)} P_1}{\langle q_0^i | P_0 | q^o \rangle \xrightarrow{\tau(?n(t))} \langle q_1^i | P_1 | q^o \rangle}\ \textsc{SER-TI} \qquad \frac{P_0 \xrightarrow{!n(t)} P_1 \quad q_1^o = n(t) \mathbin{+\!\!+} q_0^o}{\langle q^i | P_0 | q_0^o \rangle \xrightarrow{\tau(!n(t))} \langle q^i | P_1 | q_1^o \rangle}\ \textsc{SER-TO}$$

Figure 2: LTS semantics of services

Networks

Formalism: Network

$$\alpha := \tau(\gamma@n) \mid n_0 -(t)\rightarrow n_1$$

$$\frac{N(n) \xrightarrow{\tau(\gamma)} S}{N \xrightarrow{\tau(\gamma@n)} N[n \leftarrow S]}\ \textsc{NET-TAU} \qquad \frac{N(n_0) \xrightarrow{!n_1(\mathsf{C})} S_0 \quad N[n_0 \leftarrow S_0](n_1) \xrightarrow{?\bot(\mathsf{Q})} S_1}{N \xrightarrow{n_0 -(\mathsf{C})\rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]}\ \textsc{NET-CAST}$$

$$\frac{t \neq \mathsf{C} \quad N(n_0) \xrightarrow{!n_1(t)} S_0 \quad N[n_0 \leftarrow S_0](n_1) \xrightarrow{?n_0(t)} S_1}{N \xrightarrow{n_0 -(t)\rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]}\ \textsc{NET-COM}$$

Figure 3: LTS semantics of services

Monitors

Formalism: Monitors

- We define a monitored Service $\hat{\mathrm{S}}$:

$$\hat{m} :=? n(t) \mid !n(t) \mid ? n(\hat{p}) \mid !n(\hat{p})$$
$$\hat{q} := \varepsilon \mid \hat{m} \mid \hat{q} +\!\!+ \hat{q}$$
$$\hat{S} := \langle \hat{q} \mid P \mid S \rangle$$

- With Monitored service action:

$$\hat{\beta} := \beta \mid ? n(\hat{p}) \mid !n(\hat{p}) \mid \hat{\tau}(? n(t)) \mid \hat{\tau}(!n(t)) \mid \hat{\tau}(? n(\hat{p}))$$

- A monitored Service: $\left\langle \hat{q} \mid \widehat{M} \mid S \right\rangle$

Monitor algorithm function: $\hat{\mathcal{A}} : \widehat{\mathcal{M}} \times \hat{\mathfrak{m}} \to \widehat{\mathcal{M}} \times \hat{\mathfrak{q}}$

Slide

$$\frac{(\widehat{M}_1, \widehat{q}') = \widehat{\mathscr{A}}(\widehat{M}_0, ?n(t)) \quad S_0 \xrightarrow{?n(t)} S_1}{\langle ?n(t) \mathbin{+\!\!+} \widehat{q}_1 \mid \widehat{M}_0 \mid S_0 \rangle \xrightarrow{\widehat{\tau}(?n(t))} \langle \widehat{q}' \mathbin{+\!\!+} \widehat{q}_1 \mid \widehat{M}_1 \mid S_1 \rangle}\ \textsc{MON-TI}$$

$$\frac{\widehat{q}_0 = !n(t) \mathbin{+\!\!+} \widehat{q}_1 \quad (\widehat{M}_1, \widehat{q}') = \widehat{\mathscr{A}}(\widehat{M}_0, !n(t))}{\langle \widehat{q}_0 \mid \widehat{M}_0 \mid S \rangle \xrightarrow{!n(t)} \langle \widehat{q}' \mathbin{+\!\!+} \widehat{q}_1 \mid \widehat{M}_1 \mid S \rangle}\ \textsc{MON-O}$$

$$\frac{\widehat{q}_1 = \widehat{q}_0 \mathbin{+\!\!+} ?n(t)}{\langle \widehat{q}_0 \mid \widehat{M} \mid S \rangle \xrightarrow{?n(t)} \langle \widehat{q}_1 \mid \widehat{M} \mid S \rangle}\ \textsc{MON-I} \qquad \frac{\widehat{q}_0 = !n(\widehat{p}) \mathbin{+\!\!+} \widehat{q}_1}{\langle \widehat{q}_0 \mid \widehat{M} \mid S \rangle \xrightarrow{!n(\widehat{p})} \langle \widehat{q}_1 \mid \widehat{M} \mid S \rangle}\ \textsc{MON-MO}$$

$$\frac{\widehat{q}_1 = \widehat{q}_0 \mathbin{+\!\!+} ?n(\widehat{p})}{\langle \widehat{q}_0 \mid \widehat{M} \mid S \rangle \xrightarrow{?n(\widehat{p})} \langle \widehat{q}_1 \mid \widehat{M} \mid S \rangle}\ \textsc{MON-MI} \qquad \frac{\widehat{q}_0 = ?n(\widehat{p}) \mathbin{+\!\!+} \widehat{q}_1 \quad (\widehat{M}_1, \widehat{q}') = \widehat{\mathscr{A}}(\widehat{M}_0, ?n(\widehat{p}))}{\langle \widehat{q}_0 \mid \widehat{M}_0 \mid S \rangle \xrightarrow{\widehat{\tau}(?n(\widehat{p}))} \langle \widehat{q}' \mathbin{+\!\!+} \widehat{q}_1 \mid \widehat{M}_1 \mid S \rangle}\ \textsc{MON-TMI}$$

$$\frac{S_0 \xrightarrow{\tau(\beta)} S_1}{\langle \widehat{q} \mid \widehat{M} \mid S_0 \rangle \xrightarrow{\tau(\beta)} \langle \widehat{q} \mid \widehat{M} \mid S_1 \rangle}\ \textsc{MON-T} \qquad \frac{\widehat{q}_1 = \widehat{q}_0 \mathbin{+\!\!+} !n(t) \quad S_0 \xrightarrow{!n(t)} S_1}{\langle \widehat{q}_0 \mid \widehat{M} \mid S_0 \rangle \xrightarrow{\widehat{\tau}(!n(t))} \langle \widehat{q}_1 \mid \widehat{M} \mid S_1 \rangle}\ \textsc{MON-TO}$$

Fig. 9. LTS semantics of monitored services. In rule MON-T, β is either $?n(t)$, $!n(t)$, or τ (from Def. 3.4).

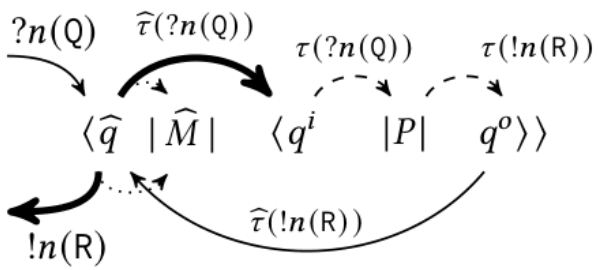

Fig. 10. Visualisation of the communications in a monitored service that immediately responds to an incoming query.

Figure 4: LTS semantics of monitored networks & visualization of communication.

Formalism: Monitored Network action

- The lts-semantics given in Figure 5 is using the transition labels $\hat{\alpha}$

$$\hat{\alpha} := \alpha \mid \hat{\tau}(\gamma@n) \mid n_0 - (\hat{p}) \to n_1$$

$$\frac{\widehat{N}(n_0) \xrightarrow{!n_1(t)} \widehat{S}_0 \quad \widehat{N}[n_0 \leftarrow \widehat{S}_0](n_1) \xrightarrow{?n_0(t)} \widehat{S}_1 \quad t \neq \mathsf{C}}{N \xrightarrow{n_0 -(t)\to n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]}\ \textsc{MNET-SCOM}$$

$$\frac{\widehat{N}(n_0) \xrightarrow{!n_1(\mathsf{C})} \widehat{S}_0 \quad \widehat{N}[n_0 \leftarrow \widehat{S}_0](n_1) \xrightarrow{?\perp(\mathsf{Q})} \widehat{S}_1}{N \xrightarrow{n_0 -(t)\to n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]}\ \textsc{MNET-SCAST}$$

$$\frac{\widehat{N}(n_0) \xrightarrow{!n_1(\widehat{p})} \widehat{S}_0 \quad \widehat{N}[n_0 \leftarrow \widehat{S}_0](n_1) \xrightarrow{?n_0(\widehat{p})} \widehat{S}_1}{\widehat{N} \xrightarrow{n_0 -(\widehat{p})\to n_1} \widehat{N}[n_0 \leftarrow \widehat{S}_0][n_1 \leftarrow \widehat{S}_1]}\ \textsc{MNET-COM}$$

$$\frac{\widehat{N}(n) \xrightarrow{\tau(\gamma)} \widehat{S}}{\widehat{N} \xrightarrow{\tau(\gamma@n)} \widehat{N}[n \leftarrow \widehat{S}]}\ \textsc{MNET-STAU}$$

$$\frac{\widehat{N}(n) \xrightarrow{\widehat{\tau}(\widehat{\beta})} \widehat{S}}{\widehat{N} \xrightarrow{\widehat{\tau}(\widehat{\beta}@n)} \widehat{N}[n \leftarrow \widehat{S}]}\ \textsc{MNET-TAU}$$

Figure 5: LTS semantics of monitored networks

Instrumentation

- Monitor instrumentation: $\mathcal{M} : \mathcal{N} \to \widehat{\mathcal{N}}$
- Deinstrumentation: $\mathcal{M}^{-1} : \widehat{\mathcal{N}} \to \mathcal{N}$

Formalism: Deadlocks Set

- Todo: Deadlock of single node
- Todo: Deadlock set

The Algorithm

- Monitor states $\widehat{M}$ is a record with the fields “probe” | “waiting” | “alarm”
- Implementation of $\hat{\mathcal{A}}$

$$\widehat{\mathscr{A}}\left(\widehat{M}, ?n^{\perp}(\mathtt{Q})\right) = \begin{cases} \left(\widehat{M},\ \epsilon\right) & \text{if } n^{\perp} = \perp \quad \textbf{ALG-CI} \\ \left(\widehat{M}[\mathtt{waiting} \leftarrow \widehat{M}.\mathtt{waiting} \cup \{n^{\perp}\}],\ \epsilon\right) & \text{if } \widehat{M}.\mathtt{probe} = \perp \quad \textbf{ALG-QI-NL} \\ \left(\widehat{M}[\mathtt{waiting} \leftarrow \widehat{M}.\mathtt{waiting} \cup \{n^{\perp}\}],\ !n(\widehat{M}.\mathtt{probe})\right) & \text{else} \quad \textbf{ALG-QI-L} \end{cases}$$

$$\widehat{\mathscr{A}}\left(\widehat{M}, !n(\mathtt{Q})\right) = \left(\widehat{M}[\mathtt{probe} \leftarrow \mathtt{freshProbe}()],\ \epsilon\right) \quad \textbf{ALG-QO}$$

$$\widehat{\mathscr{A}}\left(\widehat{M}, ?n(\mathtt{R})\right) = \left(\widehat{M}[\mathtt{probe} \leftarrow \perp],\ \epsilon\right) \quad \textbf{ALG-RI}$$

$$\widehat{\mathscr{A}}\left(\widehat{M}, !n(\mathtt{R})\right) = \left(\widehat{M}[\mathtt{waiting} \leftarrow \widehat{M}.\mathtt{waiting} \setminus \{n\}],\ \epsilon\right) \quad \textbf{ALG-RO}$$

$$\widehat{\mathscr{A}}\left(\widehat{M}, ?n(\widehat{p})\right) = \begin{cases} \left(\widehat{M}[\mathtt{alarm} \leftarrow \mathrm{true}],\ \epsilon\right) & \text{if } \widehat{M}.\mathtt{probe} = \widehat{p} \quad \textbf{ALG-P-ALARM} \\ \left(\widehat{M},\ \epsilon\right) & \text{if } \widehat{M}.\mathtt{probe} = \perp \quad \textbf{ALG-P-IGNORE} \\ \left(\widehat{M},\ \left[!n'(\widehat{p}) \ \text{ for } \ n' \in \widehat{M}.\mathtt{waiting}\right]\right) & \text{otherwise} \quad \textbf{ALG-P-PROPAGATE} \end{cases}$$

$$\widehat{\mathscr{A}}\left(\widehat{M}, !n(\mathtt{C})\right) = \left(\widehat{M},\ \epsilon\right) \quad \textbf{ALG-CO}$$

Figure 6: Implementation of $\mathscr{A}$

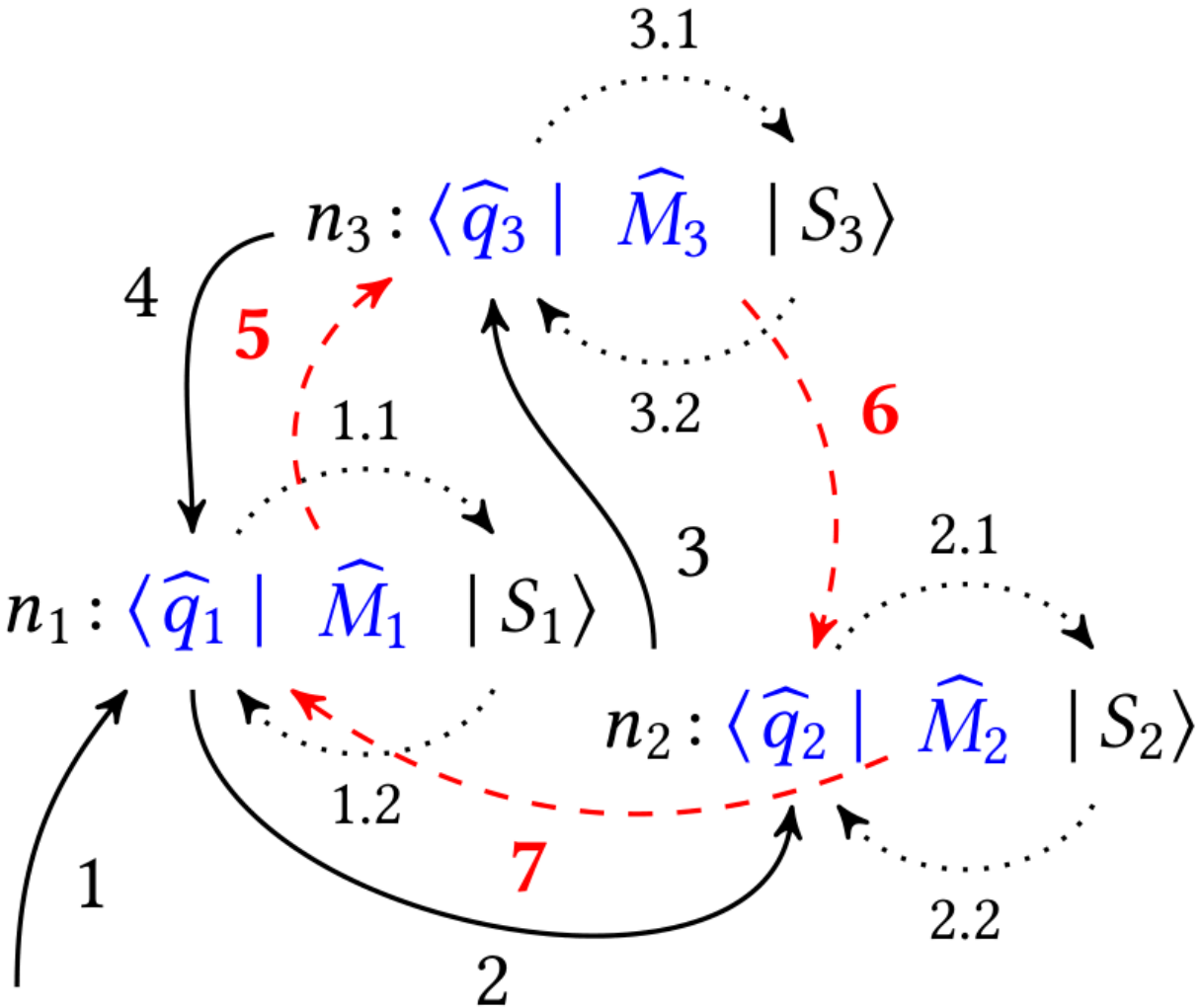

Figure 7: Deadlock detection algorithm example with three nodes.

Correctness Proofs

- Definition: Initial Network
- Definition: Client
- Definition: SRPC well-formedness
- Definition: Wellformed SRPC Network
- Theorem: Wellformedness as an *invariant*
- Definition: Complete Lock Knowledge
- Definition: Alarm Condition
- Definition: Sound lock knowledge
- Theorem: Sound lock knowledge is an *invariant*
- Theorem 6.13 (Deadlock detection preciseness)
- Theorem 6.13 (eventual deadlock reporting)

Evaluation

- Overhead is neglectible